

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering



LAB MANUAL

Course No: CSE 4120

Course Name: Data Mining Sessional

Semester: 4th Year Odd Semester

Prepared By:

Dr. Julia Rahman

Associate Professor

Department of CSE, RUET

A. F. M. Minhazur Rahman

Assistant Professor

Department of CSE, RUET

Rajshahi, Bangladesh

Data Mining Laboratory Guidelines

Before Coming to Lab

- Read the lab sheet/manual carefully before class.
- Review relevant theory: preprocessing, statistics, and Python syntax.
- Ensure Python 3 and required libraries (pandas, numpy, sklearn, seaborn, matplotlib) are installed.
- Prepare short answers or pseudocode if assigned.

During Lab Session

A. Conduct

- Be punctual and maintain professional etiquette.
- Keep your code modular and properly commented.

B. Work Process

- Perform each experiment individually or in assigned groups.
- Validate outputs and record results in notebook or Jupyter.
- Interact with instructors for clarification.

After Completing Experiment

- Demonstrate your implementation for evaluation.
- Submit a report including Objective, Theory, Code, Outputs, Discussion, and Conclusion.
- Clean up your workstation and upload files if required.

Evaluation Criteria

Criteria	Description
Preparation	Pre-lab readiness, environment setup
Implementation	Correct use of Python libraries and workflow
Understanding	Ability to explain data handling steps and logic
Reporting	Completeness and clarity of report with plots
Discipline	Attendance, punctuality, lab behavior

CO–PO Mapping for CSE 4120

CO	CO Statement	Mapped PO	Assessment Tools
CO1	Understand fundamental data-mining concepts and preprocessing methods.	PO1	Lab Performance, Report, Viva
CO2	Apply data-mining techniques to discover patterns and correlations.	PO2	Lab Performance, Report, Viva
CO3	Design classification and clustering models for predictive analysis.	PO3	Lab Performance, Report, Viva

Course Overview

Course Code: CSE 4120

Credit: 0.75

Semester: 4th Year Odd

Course Rationale

Data mining plays a vital role in extracting useful knowledge from large datasets for decision-making in business, healthcare, and engineering. This sessional provides hands-on training in data preprocessing, pattern mining, classification, clustering, and predictive analysis using modern tools and libraries.

Course Objectives

- Understand fundamental data-mining concepts, preprocessing and evaluation methods.
- Apply algorithms to discover patterns and associations in real datasets.
- Design and implement classification and clustering models.
- Evaluate results using appropriate metrics and visualizations.
- Use modern Python-based tools for data analysis.

Course Content

Introduction to Data Mining; Data Preprocessing; Mining Frequent Patterns and Associations; Classification; Clustering and Prediction; Advanced Analyses.

Experiment Plan

Week	Experiment Topic	Delivery Method	Aligned CO
1	Introduction to Data Mining and Tools	Lecture, Lab Manual 1	CO1
2	Data Preprocessing	Lecture, Lab Manual 2	CO1
3	Exploratory Data Analysis (EDA)	Lecture, Lab Manual 3	CO2
4	Mining Frequent Itemsets and Association Rules	Lecture, Lab Manual 4	CO2
5	Classification Techniques	Lecture, Lab Manual 5	CO3
6	Clustering (k-Means, Hierarchical)	Lecture, Lab Manual 6	CO3
7	Viva and Project Evaluation	Discussion, Presentation	All

Learning Materials

1. Jiawei Han, Micheline Kamber, Jian Pei, *Data Mining: Concepts and Techniques*.
2. Ian H. Witten et al., *Data Mining: Practical Machine Learning Tools and Techniques*.
3. Scikit-learn and Pandas official documentation.

Contents

Lab 1: Environment Setup and Data Loading	7
Lab 2: Data Preprocessing and Cleaning	11
Lab 3: Exploratory Data Analysis (EDA)	15
Lab 4: Mining Frequent Patterns and Association Rules	19
Lab 5: Classification Techniques	25
Lab 6: Clustering Techniques	31

Lab 1: Environment Setup and Data Loading

Objective

To set up the Python data science environment using Miniconda, install essential libraries (pandas, numpy, scikit-learn, matplotlib, seaborn), and explore initial data loading and visualization commands.

Theory

Data mining experiments rely on reproducible environments and a consistent software stack. Python, with its scientific libraries, offers a flexible and widely supported ecosystem. Conda environments allow isolated package management, avoiding version conflicts.

Tools Required

- Miniconda or Anaconda
- Python 3.10 or later
- JupyterLab / VS Code
- Libraries: pandas, numpy, scikit-learn, seaborn, matplotlib

Installation Procedure

A. Windows Installation

1. Download **Miniconda** (Windows 64-bit) from <https://docs.conda.io/en/latest/miniconda.html>
2. Run the installer → “Add Miniconda to PATH”.
3. Open **Anaconda Prompt** and execute:

```
conda create -n dm-lab python=3.11 -y
conda activate dm-lab
conda install -c conda-forge pandas numpy scikit-learn
→ matplotlib seaborn jupyterlab -y
```

4. Verify installation:

```
python -c "import pandas, numpy, sklearn, matplotlib, seaborn;  
→ print('All OK!')"
```

B. Linux / macOS Installation

1. Download installer:

```
wget  
→ https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh  
bash Miniconda3-latest-Linux-x86_64.sh  
source ~/.bashrc
```

2. Create and activate environment:

```
conda create -n dm-lab python=3.11 -y  
conda activate dm-lab
```

3. Install required packages (same as above).

Sample Code: Loading and Inspecting Data

Python Code

```
# lab1_environment.py  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# 1. Load dataset  
df = pd.read_csv("data/sample.csv") # replace with actual path  
  
# 2. Inspect top rows  
print("Head:\n", df.head())  
  
# 3. Overview of dataset  
print("\nInfo:")  
print(df.info())  
  
# 4. Statistical summary  
print("\nDescribe:")  
print(df.describe())  
  
# 5. Missing values  
print("\nMissing values per column:")  
print(df.isnull().sum())  
  
# 6. Visualization  
sns.pairplot(df.select_dtypes(include=np.number))  
plt.show()
```

Explanation of Key Functions

- `pd.read_csv()`: reads CSV data into DataFrame.
- `df.head(x)`: displays top x records.
- `df.info()`: shows column types and non-null counts.
- `df.describe()`: returns statistical summary.
- `df.isnull().sum()`: checks missing values.
- `sns.pairplot()`: visualizes pairwise relationships between numerical features.

Discussion

Proper environment setup ensures reproducibility and minimizes dependency issues. Using pandas and seaborn, students can quickly perform exploratory data analysis (EDA), which is foundational to all later labs on preprocessing, pattern mining, and model building.

Exercises

1. Create the conda environment `dm-lab` and install required packages.
2. Load a public dataset (e.g., *Iris* or *Titanic*) and display its `head()`, `info()`, and `describe()`.
3. Identify columns with missing values and report them.
4. Produce at least one visualization using matplotlib or seaborn and interpret it.

References

- <https://docs.conda.io/>
- <https://pandas.pydata.org/docs/>
- <https://seaborn.pydata.org/>

Lab 2: Data Preprocessing and Cleaning

Objective

To understand and implement basic data preprocessing techniques such as handling missing values, encoding categorical features, normalization, and feature scaling using `pandas` and `scikit-learn`.

Theory

Raw data collected from real-world sources often contains noise, missing entries, or inconsistent formats. Data preprocessing aims to clean and transform such data into a structured form suitable for model training. Common preprocessing tasks include:

- Handling missing or null values.
- Encoding categorical data into numeric form.
- Normalizing or scaling numerical attributes.
- Removing duplicate or irrelevant records.

Major Functions Used

- `df.isnull().sum()`: Counts missing values per column.
- `df.fillna(value)`: Fills missing values with a constant, mean, or mode.
- `df.dropna()`: Removes rows or columns with missing data.
- `pd.get_dummies()`: Performs one-hot encoding for categorical variables.
- `StandardScaler()`, `MinMaxScaler()`: From `sklearn.preprocessing`, used for normalization and scaling.

Procedure

1. Load Data and Inspect

```
1 import pandas as pd
2 import numpy as np
3
4 df = pd.read_csv("data/dirty_data.csv")
5
6 print("Initial shape:", df.shape)
7 print("\nMissing values:\n", df.isnull().sum())
8 print("\nData types:\n", df.dtypes)
```

2. Handle Missing Values

```
1 # Option 1: Drop rows with any missing value
2 df_drop = df.dropna()
3
4 # Option 2: Fill missing values
5 df_fill = df.fillna({
6     'Age': df['Age'].mean(),
7     'Salary': df['Salary'].median(),
8     'City': 'Unknown'
9 })
```

Explanation: Dropping removes incomplete data but may lead to information loss. Filling preserves data count and is preferred when missing proportion is small.

3. Encode Categorical Data

```
1 # One-hot encoding using pandas
2 df_encoded = pd.get_dummies(df_fill, columns=['City', 'Gender'],
3                               drop_first=True)
4
5 print(df_encoded.head())
```

Explanation: Converts categorical variables like “Male/Female” or “Dhaka/Rajshahi” into binary columns, allowing numerical algorithms to process them.

4. Normalize and Scale Numeric Columns

```
1 from sklearn.preprocessing import StandardScaler, MinMaxScaler
2
3 num_cols = ['Age', 'Salary']
4
5 # Standardization: mean = 0, std = 1
6 scaler_std = StandardScaler()
7 df_std = df_encoded.copy()
8 df_std[num_cols] = scaler_std.fit_transform(df_std[num_cols])
9
10 # Normalization: range [0, 1]
11 scaler_minmax = MinMaxScaler()
12 df_norm = df_encoded.copy()
13 df_norm[num_cols] = scaler_minmax.fit_transform(df_norm[num_cols])
```

Explanation: Scaling ensures that all features contribute equally to the model and prevents bias toward larger numeric values.

5. Remove Duplicates

```
1 before = df_norm.shape[0]
2 df_clean = df_norm.drop_duplicates()
3 after = df_clean.shape[0]
4 print(f"Removed {before - after} duplicate rows")
```

6. Full Example Script

```
1 import pandas as pd
2 from sklearn.preprocessing import StandardScaler
3
4 # Load data
5 df = pd.read_csv("data/dirty_data.csv")
6
7 # Fill missing values
8 df.fillna({
9     'Age': df['Age'].mean(),
10    'Salary': df['Salary'].median(),
11    'City': 'Unknown'
12 }, inplace=True)
13
14 # Encode categorical columns
15 df = pd.get_dummies(df, columns=['City', 'Gender'], drop_first=True)
16
17 # Standardize numeric columns
18 scaler = StandardScaler()
19 num_cols = ['Age', 'Salary']
20 df[num_cols] = scaler.fit_transform(df[num_cols])
21
22 # Drop duplicates
23 df.drop_duplicates(inplace=True)
24
25 print(df.head())
```

Listing 1: Complete Preprocessing Example

Discussion

Proper data preprocessing is essential before applying any data mining or machine learning algorithm. Inconsistent or unscaled data leads to incorrect model training, while imputation and normalization improve accuracy and convergence.

Exercises

1. Load a dataset with missing and categorical values (e.g., Titanic dataset).
2. Try three imputation methods: mean, median, and mode. Compare their effects.
3. Apply one-hot encoding and observe the change in column count.
4. Normalize numeric columns using both `StandardScaler` and `MinMaxScaler`.
5. Write a short report explaining how preprocessing impacted your dataset.

Outcome

Students will gain hands-on experience cleaning datasets, applying imputation, encoding, and scaling—crucial preprocessing steps for all subsequent labs.